หลักไล่ผลการรัน (ใช้กับทุกบท)

- **new ลูก()** → constructor **แม่ก่อน** แล้วค่อยลูก (Java แอบใส่ `super()` ให้ถ้าไม่เขียน)
- `Parent p = new Child();` **ชนิดตัวแปร** = เรียกเมธอดอะไรได้ (ตรวจตอน compile) · **ชนิด object จริง** = ใช้เวอร์ชันไหน (runtime)
- **field และ static ไม่ polymorphic**: `p.name` ได้ค่าของ Parent · `p.show()` ได้ของ Child ถ้า override
- ในเมธอดลูกเจอ `super.m()` → รันของแม่ ณ จุดนั้น แล้วทำบรรทัดถัดไปของลูก
- try: พังแล้ว**ข้ามที่เหลือใน try** → catch ตัว**แรกที่ตรงชนิด** → **finally ทำเสมอ** → ทำต่อหลังบล็อก
- `7/2=3` · `(double)7/2=3.5` · `10.0/0=Infinity` (ไม่ error) · `10/0` → ArithmeticException
- `printf("%.2f%n", x)` = ทศนิยม 2 ตำแหน่ง + ขึ้นบรรทัด · `print` ไม่ขึ้นบรรทัด · `"a"+1+2="a12"` · `1+2+"a"="3a"`

Compile error ที่ชอบออก: new abstract class/interface · เรียกเมธอดที่ชนิดตัวแปรไม่มี · `super()` ไม่ใช่บรรทัดแรก · แม่ไม่มี constructor เปล่าแต่ลูกไม่เรียก super(..) · catch(Exception) อยู่ก่อน catch ลูก · ไม่ override abstract ให้ครบ (และลูกไม่ abstract) · implement interface แต่ไม่ใส่ public · override แคบลง · overload ต่างแค่ return type · `ArrayList<int>` · กำหนดค่าให้ final/ค่าคงที่ใน interface · ไม่จัดการ checked exception
Runtime error: AIOOBE, NPE, ClassCastException, NumberFormat, Arithmetic, InputMismatch, ConcurrentModification

บท 7 · Array & ArrayList

```
int[] a = new int[5];            // 0 ทุกช่อง, index 0..4
int[] b = {75,88,92,60,78};      // แนะนำ
int[] c = new int[]{75,88,92};
for(int i=0;i<b.length;i++) sum+=b[i];   // < ห้าม <=
for(int x : b) ...               // for-each: อ่านอย่างเดียว
double avg = (double)sum / b.length;     // cast!
```

ค่าเริ่มต้น	
int/long/short/byte · double/float	0 · 0.0
boolean · char	false · '\u0000'
String / object ทุกชนิด	**null**

- `a.length` **ไม่มี ()** · `str.length()` มี () · `list.size()`
- index สุดท้าย = `length-1` · a[5] บน size 5 → **ArrayIndexOutOfBoundsException**
- เทียบ String ใช้ `.equals()` ไม่ใช่ `==`
- หาค่ามาก: `max=a[0]; for(i=1..) if(a[i]>max) max=a[i];`

2 มิติ [แถว][คอลัมน์]

```
int[][] s = new int[3][4];       // 3 แถว 4 คอลัมน์
int[][] s = {{75,80,70},{88,70,95},{92,85,88}};
s.length = 3 แถว   s[0].length = 3 คอลัมน์   s[1][2] = 95
for(int r=0;r<s.length;r++)
  for(int c=0;c<s[r].length;c++) ...  // jagged ใช้ s[r].length
for(int[] row : s) for(int v : row) ...
```

รวมรายแถว: reset total ทุกแถว · เฉลี่ยรายคอลัมน์: ลูปนอก col ลูปใน row · `Subject[][] t = new Subject[3][3];`

Array of Objects (เก็บ reference)

```
Student[] st = new Student[3];   // null 3 ช่อง ยังไม่มี object!
st[0].showInfo();                //
  NullPointerException
st[0] = new Student("สมชาย",3.25); // ต้อง new ทีละตัว
Student[] st = {new Student("A",3.1), new Student("B",3.6)};
if(st[i]!=null) st[i].showInfo();
Student top=st[0]; for(i=1..)
  if(st[i].getGpa()>top.getGpa()) top=st[i];
```

ArrayList (import java.util.ArrayList;)

```
ArrayList<String> n = new ArrayList<>();
n.add("A"); n.add("B"); n.add("C");   // [A, B, C]
n.add(1,"X");      // แทรก → [A, X, B, C]
n.get(2);          // "B"      n.size();  // 4
n.set(0,"Z");      // [Z, X, B, C]
n.remove(0);       // เลื่อนซ้าย → [X, B, C]
n.remove("B");     // ลบตามค่า → [X, C]
n.contains("X");  // true    n.indexOf("C"); // 1
n.isEmpty(); n.clear();  System.out.println(n); // [X, C]
ArrayList<Integer> li = [10,20,30]: li.remove(1) → [10, 30] (index!)
  li.remove(Integer.valueOf(10)) → [30] (ตามค่า)
```

Array	ArrayList
ขนาดคงที่ · `.length`	ยืดหดได้ · `.size()`
primitive ได้	ต้อง wrapper: Integer Double Character Boolean (autoboxing)
ลบกลางต้อง shift เอง	remove แล้ว shift ให้อัตโนมัติ

ลบใน for-each → ConcurrentModificationException · ใช้วนถอยหลัง `for(int i=list.size()-1;i>=0;i--) if(..) list.remove(i);` · `get(list.size())` เกิน · ลืม import · `<int>` ผิด

บท 8 · Inheritance

- **IS-A** → `extends` (Student is a Person, Dog is a Animal) · **HAS-A** → attribute/composition (Car has Engine, CafeOrder has ArrayList<OrderItem>)
- Superclass = คลาสแม่ · Subclass = คลาสลูก · extends ได้ **1 คลาส** · ลูกได้ attribute+method ของแม่ (ยกเว้น private เข้าตรงไม่ได้, constructor ไม่สืบทอด)
- ประโยชน์: code reuse · maintainability · extensibility · จำลองโลกจริง

modifier	class	package	ลูก	ทุกที่	UML
public	✓	✓	✓	✓	+
protected	✓	✓	✓	×	#
(default)	✓	✓	×*	×	~
private	✓	×	×	×	–

*ลูกที่อยู่ต่าง package

```
class Student extends Person {
  private String studentId;
  public Student(String name,int age,String id){
    super(name, age);          // ต้องบรรทัดแรก
    this.studentId = id;
  }
  @Override public void showInfo(){
    super.showInfo();          // ของแม่ก่อน
    System.out.println("ID: "+studentId);
  }
}
```

กฎ Override

- ชื่อ + พารามิเตอร์ (ชนิด/ลำดับ/จำนวน) **เหมือนเดิม** · return เหมือนหรือ subtype (covariant)
- access **ห้ามแคบลง** (private<default<protected<public)
- **override ไม่ได้**: static (= method hiding), final, private, constructor
- `@Override` ไม่บังคับ แต่ถ้าพิมพ์ชื่อผิด (makesound) จะ compile error ช่วยจับ
- ไม่เรียก super.m() = ทับพฤติกรรมแม่ทั้งหมด · `base=super.calculatePay(); return base+bonus;`
- Multi-level: Animal → Dog → Poodle (ได้ของทั้งสองชั้น)

บท 9 · Abstraction

ซ่อนรายละเอียด แสดงเฉพาะที่จำเป็น · 2 เครื่องมือ: **abstract class**, **interface**

```
public abstract class Shape {
  private String color;
  public Shape(String color){ this.color=color; }
    // มี ctor ได้
  public abstract double calculateArea();  // ไม่มี body ;
  public void displayInfo(){
    System.out.println("Color: "+color);
    System.out.printf("Area: %.2f%n",
      calculateArea());
  }
}
class Circle extends Shape {
  private double radius;
  public Circle(String color,double r){
    super(color); radius=r; }
  @Override public double calculateArea(){ return
    Math.PI*radius*radius; }
}
Shape s = new Shape("r");       // ✗
Shape s = new Circle("r",5);    // ✓
```

- มี abstract method ≥1 → class **ต้อง** abstract · abstract class ไม่มี abstract method เลยก็ได้
- abstract method ห้าม private/final/static · ลูกต้อง override **ครบ** ไม่งั้นลูกต้อง abstract (Shape→abstract RoundShape→Circle)
- UML: ชื่อตัวเอียง/«abstract», method `{abstract}`

```
interface PaymentMethod { void pay(double amount); }
class CashPayment implements PaymentMethod {
  @Override public void pay(double amount){   // ต้อง public
    System.out.printf("Paid %.2f baht by cash%n", amount);
  }
}
class Phone implements Camera, MusicPlayer, GPS {}
class FullTime extends Employee implements Taxable {} // extends ก่อน
interface MFD extends Printable, Scannable { void copy(); }
```

- ใน interface: method = `public abstract` อัตโนมัติ · ตัวแปร = `public static final` (ค่าคงที่, แก้ไม่ได้: `Taxable.VAT_RATE=0.1` ×) · มี `default` (มี body), `static` (เรียก `Calculator.square(5)`), `private`
- UML «interface» + **เส้นประ** หัวสามเหลี่ยมโปร่ง = implements · เส้นทึบ = extends

	abstract class	interface
ใช้	extends (1)	implements (หลาย)
new	×	×
constructor	✓	×
instance field	✓	× (มีแต่ค่าคงที่)
method ปกติ	✓	default/static/private
access	ได้ทุกแบบ	abstract = public
เหมาะกับ	is-a ญาติใกล้, แชร์ field/ctor	ความสามารถร่วม คลาสไม่เกี่ยวกัน (-able)

Abstraction = ทำอะไรได้ (abstract/interface) · **Encapsulation** = ปกป้องข้อมูล (private + getter/setter) · 4 เสา: Abstraction, Encapsulation, Inheritance, Polymorphism

```
interface Taxable { double TAX_RATE = 0.05; double calculateTax(); }
public double calculateTax(){ return calculateSalary()*TAX_RATE; }
```

บท 10 · Polymorphism

	Overloading	Overriding
ที่ไหน	class เดียวกัน	แม่–ลูก / interface
พารามิเตอร์	**ต่าง**	**เหมือน**
return	ต่างได้ (แต่ต่างแค่ return = **error**)	เหมือน/covariant
ตัดสินตอน	compile (static/early binding)	runtime (dynamic dispatch/late)
ตัวอย่าง	add(int,int) add(double,double)	animal.makeSound()

```
Animal a = new Dog();   // upcasting อัตโนมัติ ปลอดภัย
a.makeSound();          // Dog barks: Woof!
a.guardHouse();         // ✗ compile error (Animal ไม่มี)
Dog d = (Dog) a;        // downcasting ต้อง cast เอง
Animal x = new Cat(); Dog y = (Dog) x; // ClassCastException (runtime)
if (a instanceof Dog dog) dog.guardHouse(); // เช็คก่อน
Animal[] zoo = {new Dog(), new Cat(), new Cow()};
for (Animal z : zoo) z.makeSound();  // แต่ละตัวต่างกัน
ArrayList<Shape> list; total += s.calculateArea();
// Circle(5)=78.54 Rect(4,6)=24.00
  Tri(8,3)=0.5*b*h=12.00 → 114.54
```

- ประโยชน์: ลด if-else/switch · เพิ่มคลาสใหม่ไม่ต้องแก้ลูป (**Open/Closed**) · **program to an interface** `checkout(PaymentMethod p)` · loose coupling
- instanceof เยอะ ๆ = ใช้ polymorphism ไม่ดี
- Employee: FullTime(30000+5000)=35000 · PartTime(80h×200)=16000 · รวม 51000

แม่แบบ 2.1 · UML → โค้ด

+public #protected –private ~default · `name : String` → `String name;` · `m(x:int) : double` → `double m(int x)` · ตัวเอียง/«abstract»→abstract · ขีดเส้นใต้→static · เส้นทึบ▷extends · เส้นประ▷implements · ลูกศรเส้นทึบธรรมดา/◇ = has-a (field)

```
public abstract class Pet {
  protected String name;
  public Pet(String name){ this.name = name; }
  public abstract void makeSound();
  public void showPetInfo(){
    System.out.print(name+" Making Sound : ");
    makeSound(); }
}
public class Dog extends Pet {
  public Dog(String name){ super(name); }
  @Override public void makeSound(){
    System.out.println("Bok Bok..."); }
}  // Cat: "Meaow Meaow..." · Bird เหมือนกัน
public abstract class Employee {
  private String employeeId, name;
  public Employee(String id, String name){
    employeeId=id; this.name=name; }
  public abstract double calculateSalary();
  public void displayInfo(){
    System.out.println("ID: "+employeeId+" Name: "+name
      +" Salary: "+calculateSalary()); }
}
public class FullTimeEmployee extends Employee {
  private double baseSalary, bonus;
  public FullTimeEmployee(String id,String n,double base,double bonus){
    super(id, n); this.baseSalary=base; this.bonus=bonus; }
  @Override public double calculateSalary(){
    return baseSalary + bonus; }
}  // PartTime: hoursWorked * hourlyRate
public class Main { public static void main(String[] args){
  ArrayList<Pet> pets = new ArrayList<>();
  pets.add(new Dog("Buddy")); pets.add(new Cat("Mimi"));
  for (Pet p : pets) p.showPetInfo();
} }
```

ขั้นตอน: แม่ (field→ctor→abstract/method) → ลูก (extends, ctor+super บรรทัดแรก, @Override ครบ) → main (ArrayList<แม่> + for-each) · อย่าลืม import, ;, public ใน implements

หน้า 2: บท 11–12 + แม่แบบตอนที่ 2.2

ดูช่องว่าง → เดาจากคำรอบ ๆ · parse → NumberFormatException · หาร → ArithmeticException · Scanner → InputMismatchException

บท 11 · Exception Handling

ข้อผิดพลาด	ลักษณะ
Syntax/Compile	ลืม ; ชื่อผิด → compiler หยุด ไม่ได้รัน
Runtime (Exception)	รันแล้วพังกลางทาง (หาร 0, index เกิน)
Logical	รันได้ แต่ผลผิด

Exception	เกิดจาก
ArithmeticException	`10/0` → msg "/ by zero"
ArrayIndexOutOfBounds	`score[5]` บน {80,70,90}
NumberFormatException	`Integer.parseInt("ABC")`, `Double.parseDouble("")`
NullPointerException	`String n=null; n.length()`
InputMismatchException	`sc.nextInt()` พิมพ์ "ABC" (import java.util)
ClassCastException	downcast ผิดชนิด
IllegalArgumentException	เรา throw เองเมื่ออาร์กิวเมนต์ผิด
FileNotFound / IOException	**checked**: new FileReader("data.txt")

ลำดับชั้น: Throwable → Error | Exception → RuntimeException (unchecked: Arithmetic, NullPointer, ...) · `catch(Exception e)` จับได้ทุกลูก (polymorphism)

```
try {
  int[] x = {10,20,30};
  System.out.println(x[5]);   // พังตรงนี้
  int r = 10/0;               // ไม่ถูกรัน
} catch (ArithmeticException e) {
  System.out.println("A");
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("I");
} finally { System.out.println("F"); }
System.out.println("End");    // ผล: I F End
```

- catch **เจาะจงก่อน กว้างทีหลัง** (Exception ท้ายสุด) · ไม่งั้น compile error
- `e.getMessage()` → `/ by zero` · `println(e)` → `java.lang.ArithmeticException: / by zero` · `e.printStackTrace()`
- ไม่มี catch ที่ตรง → finally ยังทำ แล้วโปรแกรมพัง (บรรทัดหลังไม่ทำ)
- **Unchecked** compiler ไม่บังคับ · **Checked** บังคับ try-catch หรือ throws
- **throw** = โยนทันที (ในเนื้อเมธอด) · **throws** = ประกาศที่หัวเมธอด ให้ผู้เรียกจัดการ

```
public Student(String name,int age){
  if(age<0) throw new
    IllegalArgumentException("Age cannot be
    negative");
  this.name=name; this.age=age;
}
class InsufficientBalanceException extends
   Exception {
  public InsufficientBalanceException(String
    message){ super(message); }
}
public void withdraw(double amount) throws
   InsufficientBalanceException {
  if(amount>balance) throw new
    InsufficientBalanceException("Insufficient
    balance");
  balance -= amount;
}
try { acc.withdraw(500); } catch
   (InsufficientBalanceException e) {
  System.out.println(e.getMessage()); }
```

เครื่องคิดเลข Scanner (แม่แบบ)

```
import java.util.Scanner; import
   java.util.InputMismatchException;
Scanner sc = new Scanner(System.in);
try {
  System.out.print("Enter number 1: "); int a =
    sc.nextInt();
  System.out.print("Enter number 2: "); int b =
    sc.nextInt();
  System.out.println("Result = " + (a / b));
} catch (ArithmeticException e) {
  System.out.println("Error: Cannot divide by
    zero");
} catch (InputMismatchException e) {
  System.out.println("Error: Please enter a
    number");
} finally { sc.close(); }
System.out.println("End Program");
// 10,2→Result = 5 · 10,0→divide error · ABC→ไม่ถาม
   เลข 2 → number error
```

Grade: คะแนนนอก 0–100 → `throw new IllegalArgumentException("Score must be between 0 and 100");`

บท 12 · Java GUI (Swing)

AWT (เก่า) → **Swing** (javax.swing, ใน JDK, OOP+event-driven) → JavaFX (CSS/animation)

```
import javax.swing.*; import java.awt.*;
import java.awt.event.*;   // ActionEvent,
   ActionListener
public class MyGUI extends JFrame {
  public MyGUI(){
    setTitle("My First GUI");
    setSize(500, 300);                    //
      กว้าง, สูง

      setDefaultCloseOperation(JFrame.EXIT_ON_CLOS
      E);
    setLocationRelativeTo(null);          //
      กลางจอ
  }
  public static void main(String[] args){
    SwingUtilities.invokeLater(() -> new
      MyGUI().setVisible(true));
  }  // EDT · setVisible(true) ท้ายสุด
}
```

Component	เมธอดสำคัญ
JLabel	`new JLabel("txt", SwingConstants.CENTER)` setText · `setHorizontalAlignment(SwingConstants.CENTER)`
JTextField(20)	20=ความกว้าง (คอลัมน์) **ไม่ใช่**จำนวนอักษรสูงสุด · `getText().trim()` · `setText("")` · `requestFocus()`
JPasswordField	รหัสผ่าน
JTextArea(5,20)	แถว,คอลัมน์ · `setLineWrap(true)` `setWrapStyleWord(true)` `setEditable(false)` · ใส่ `new JScrollPane(area)` ถึงเลื่อนได้ · "\n"
JButton	`addActionListener` (ไม่ผูก = กดไม่ทำอะไร)
JCheckBox	เลือกหลายข้อ · `isSelected()` `setSelected(true)`
JRadioButton	เลือกข้อเดียว · **ต้อง ButtonGroup** `g.add(male)` + ยัง add ลง panel
JComboBox<String>	`new JComboBox<>(arr)` · `(String) getSelectedItem()` · `getSelectedIndex()` · `setSelectedIndex(0)`
JTable	`new JTable(Object[][] data, String[] cols)` ใน JScrollPane (ไม่งั้นไม่เห็นหัวตาราง)
DefaultTableModel	import javax.swing.table · `new DefaultTableModel(cols,0)` · `addRow(Object[])` · `removeRow(i)` · `table.getSelectedRow()` = **-1** ถ้าไม่เลือก
JMenuBar/JMenu/JMenuItem	`menu.add(item)` `addSeparator()` `bar.add(menu)` `setJMenuBar(bar)`
ImageIcon	`new JLabel(new ImageIcon("images/a.png"))` ใช้ relative path

Layout	ลักษณะ
FlowLayout	ซ้าย→ขวา ไม่พอขึ้นบรรทัดใหม่ · ค่าเริ่มต้น JPanel · `FlowLayout.LEFT`
BorderLayout(h,v)	NORTH SOUTH EAST WEST CENTER · ค่าเริ่มต้น JFrame · `add(p, BorderLayout.NORTH)`
GridLayout(r,c,h,v)	ตารางช่องเท่ากัน (3,2,5,5)=3แถว 2คอลัมน์ ห่าง 5
BoxLayout/GridBag	แนวเดียว / ยืดหยุ่น

JPanel จัดกลุ่ม + ผสม layout (Grid ใน NORTH ของ Border) · `pack()` ปรับขนาดอัตโนมัติ ดีกว่า setSize · **ไม่แนะนำ** `setLayout(null)+setBounds` · `dispose()` ทำลาย/คืนหน่วยความจำ vs `setVisible(false)` ซ่อน

Event: Source (JButton) → Event Object (ActionEvent) → Listener (ActionListener)

```
btn.addActionListener(new ActionListener(){
  @Override public void
    actionPerformed(ActionEvent e){
    resultLabel.setText("คุณคลิกปุ่มแล้ว"); }
});
btn.addActionListener(e -> calculate());  //
  lambda: 1 เมธอด
combo.addItemListener(e -> {
  if(e.getStateChange()==ItemEvent.SELECTED) ...
  });
label.addMouseListener(new MouseAdapter(){ public
  void mouseClicked(MouseEvent e){..} });
field.addKeyListener(new KeyAdapter(){ public void
  keyReleased(KeyEvent e){..} });
```

Adapter = ไม่ต้อง override เมธอดที่ไม่ใช้ · DocumentListener ดีกว่า KeyListener (จับการวางด้วยเมาส์)

JOptionPane

```
JOptionPane.showMessageDialog(null, "บันทึกข้อมูล
  เรียบร้อย");
JOptionPane.showMessageDialog(this, "ไม่พบข้อมูล",
  "Warning", JOptionPane.WARNING_MESSAGE);
String name = JOptionPane.showInputDialog("กรุณา
  กรอกชื่อ"); // Cancel → null
if (name != null && !name.trim().isEmpty()) ...
int ans = JOptionPane.showConfirmDialog(null,
  "ออก?", "Confirm", JOptionPane.YES_NO_OPTION);
if (ans == JOptionPane.YES_OPTION) System.exit(0);
```

ชนิด: INFORMATION_ · WARNING_ · ERROR_ · QUESTION_ · PLAIN_MESSAGE

บั๊ก: ไม่เห็นหน้าต่าง→ลืม setVisible(true) · ทับกัน→ใช้ JPanel/Layout · กดแล้วเงียบ→ลืม addActionListener · ค้าง→งานหนักบน EDT ใช้ SwingWorker · พิมพ์อักษรพัง→ไม่ catch NumberFormatException

MVC: แยก Model (คลาสข้อมูล ไม่รู้จัก UI) ออกจาก GUI · GUI: รับข้อมูล → validate → สร้าง Model → แสดงผล/รับ event

```
Student st = new Student(idField.getText().trim(),
  nameField.getText().trim(),
                       (String)
                         majorCombo.getSelectedI
                         tem());
Shape shape;
  if(combo.getSelectedItem().equals("Circle"))
  shape=new Circle(r);
else shape=new Rectangle(w,h);  double area =
  shape.calculateArea();
```

BMI = kg/m² : <18.5 Under · 18.5–24.9 Normal · 25–29.9 Over · ≥30 Obese (65/170→22.49) · Grade: 80 A, 70 B, 60 C, 50 D, <50 F · CafeOrder `ArrayList<OrderItem>`; change = received – total (น้อยกว่า → "Insufficient payment")

แม่แบบ 2.2 · GUI + try-catch (AdditionGUI)

```
public class AdditionGUI extends JFrame {
  private JTextField firstField, secondField;
    private JLabel resultLabel;
  public AdditionGUI(){
    firstField = new JTextField(); secondField =
      new JTextField();
    JButton calc = new JButton("คำนวณ"), clear =
      new JButton("ล้างข้อมูล");
    resultLabel = new JLabel("ผลลัพธ์จะแสดงที่นี่",
      SwingConstants.CENTER);
    JPanel form = new JPanel(new GridLayout(2, 2,
      5, 5));
    form.add(new JLabel("จำนวนที่ 1:"));
      form.add(firstField);
    form.add(new JLabel("จำนวนที่ 2:"));
      form.add(secondField);
    JPanel btns = new JPanel(); btns.add(calc);
      btns.add(clear);
    setLayout(new BorderLayout(10, 10));
    add(form, BorderLayout.NORTH);
      add(resultLabel, BorderLayout.CENTER);
    add(btns, BorderLayout.SOUTH);
    calc.addActionListener(e -> calculate());
    clear.addActionListener(e -> clearForm());
    setTitle("โปรแกรมบวกเลข"); setSize(400, 220);

      setDefaultCloseOperation(JFrame.EXIT_ON_CLOS
      E); setLocationRelativeTo(null);
  }
  private void calculate(){
    try {
      double a =
        Double.parseDouble(firstField.getText().tr
        im());
      double b =
        Double.parseDouble(secondField.getText().t
        rim());
      resultLabel.setText("ผลลัพธ์ = " + (a + b));
    } catch (NumberFormatException e) {
      JOptionPane.showMessageDialog(this, "กรุณา
        กรอกข้อมูลเป็นตัวเลข",
          "ข้อมูลไม่ถูกต้อง",
            JOptionPane.ERROR_MESSAGE);
    }
  }
  private void clearForm(){
    firstField.setText("");
    secondField.setText("");
    resultLabel.setText("ผลลัพธ์จะแสดงที่นี่");
      firstField.requestFocus(); }
}
```

Validation: `if(id.isEmpty()||name.isEmpty()){ JOptionPane.showMessageDialog(this,"กรุณากรอกรหัสนักศึกษาและชื่อ","ข้อมูลไม่ครบ",JOptionPane.WARNING_MESSAGE); return; }` · radio: `if(maleRadio.isSelected()) gender="ชาย"; else if(...)` · `if(!agreeCheckBox.isSelected())`

ทายผลเร็ว (รันจริงแล้ว)

โค้ด	ผล
`new int[3]`→a[1] · `new String[2]`→s[0]	0 · null
[A,B,C] add(1,"X") → get(2), size()	B, 4
s={{75,80,70},{88,70,95},..} s[1][2]	95
new B() (A,B พิมพ์ใน ctor) แล้ว obj.hi() ที่ B เรียก super.hi()	A B hiA hiB
Parent o=new Child(); o.name / o.showName()	Parent / Child method
Animal a=new Cat(); (Dog)a	ClassCastException
try{10/0;}catch(Arith){"Error"}finally{"Finally"}	Error → Finally
x[5] ก่อน 10/0, catch A ก่อน I + finally F + End	I F End
catch(Exception) ก่อน catch(Arithmetic)	compile error
showInputDialog แล้วกด Cancel	null
table.getSelectedRow() ไม่ได้เลือก	-1